Werewolf - How to be a Better Villager

Chew Jin Cheng*
j.c.chew@lancaster.ac.uk
Lancaster University
37041332
Lancaster, United Kingdom

Duong Minh Hoang*
m.h.duong@lancaster.ac.uk
Lancaster University
36988202
Lancaster, United Kingdom

Hoang Phuong Nam*
p.hoang@lancaster.ac.uk
Lancaster University
37071495
Lancaster, United Kingdom

Nguyen Chi Vinh Nhan*
c.v.nguyen@lancaster.ac.uk
Lancaster University
37046901
Lancaster, United Kingdom

Vuong Thanh Cong†*
t.c.vuong@lancaster.ac.uk
Lancaster University
36997393
Lancaster, United Kingdom



Figure 1. Simulation of Language Model based Werewolf game Multi-agent System (generated by Gemini Banana Pro 3.1)

Abstract

This study investigates whether smaller language models (SLMs) can improve their performance when guided by larger language model (LLMs) in a multi-agent game. As the current research on how SLMs perform when supported by LLMs is limited, the study focuses on integrating a Coach-Player framework into the Werewolf game. In this study, villager agents are supported by a coach to play against the wolf agents to evaluate the impact of guidance on game performance. The results indicate that while the models with lower capability show no improvement, using coaching improves the performance for higher ones by increasing win rate from 55% to 74%. These findings suggest that as long as the model is able to interpret and apply the guidance, coaching can improve its performance.

1 Introduction

As technology advances, Multi-agent Systems (MAS) has become an important area in Artificial Intelligence (AI). Instead of relying on a single model, MAS involves distributing tasks to multiple agents, each with its own reasoning and behaviour, interacting with each other within an environment. This allows them to address complicated issues that are difficult for a single model to solve. The rise of LLMs has also introduced new possibilities for MAS, where LLMs can be used as agents with reasoning and complex task processing abilities.

Previous studies [42] highlighted that LLMs tend to demonstrate stronger performance in reasoning and complex tasks, while SLMs use fewer resources but are weaker in strategic planning. On the other hand, most existing work on Werewolf based experiments [36, 52, 54] shows limited to no analysis of how smaller models perform when guided by larger model in interactive multi-agent environments. This raises the question of whether the performance

*Authors contributed equally to this work.
†Corresponding author

of smaller models can be improved through guidance from larger models.

To examine this, this research introduces a Coach–Player framework within a social deduction game Werewolf. Werewolf is a multiplayer game where players hide their identities and are divided into two opposing groups: Villagers and Werewolves. In the game, Villagers have to identify and vote to eliminate the Werewolves, while the Werewolves have to eliminate the villagers. The game progresses through multiple rounds of discussion and actions, requiring players to make decisions based on limited and uncertain information. This setting of incomplete and asymmetric information provides a suitable environment for analysing how agents reason and adapt their strategies over time. By integrating the Coach-Player structure into the game, the Coach will guide the Villagers against Werewolves, allowing to observe if the performance improves when the Villagers are supported by a stronger model over multiple rounds of gameplay.

The objectives of the research are to address the following questions:

- Research Question 1 (RQ1): Can the Coach–Player paradigm effectively enhance the performance of language model agents by improving villager-side strategies?
- Research Question 2 (RQ2): Which types of models are able to effectively leverage this paradigm?

Throughout the study, all experiments will be conducted within the Werewolf game in a simulated environment, using a fixed set of model sizes. This allows to study the impact of guidance from a more advanced model on a less advanced model in a controlled scenario. Additionally, it also explores a more cost-effective setup where the smaller models handle most tasks while the larger models provide guidance only when necessary, potentially reducing overall computational cost.

Our findings show that coaching improves performance for higher-capacity models but provides little to no effect on smaller models. This disparity may be attributed to the superior capacity of larger models to leverage guidance for performance optimization, whereas smaller models frequently exhibit difficulty in accurately interpreting and integrating such feedback.

2 Related Work

2.1 Coach-Player paradigm

The Coach–Player paradigm organises multi-agent and reinforcement learning (RL) around an asymmetric relationship: a guiding agent (Coach) helps a learning agent (Player) acquire skills faster and make better decisions. The idea has roots in educational AI [5] found that agents assigned a teaching role strengthen their own knowledge in the process, the learning-by-teaching effect. [40] brought this into RL with a teacher–student framework where the teacher advises the student on actions under a finite budget; the key finding is that when advice is given matters more than how much. Later work split in two directions: peer-to-peer advising in multi-agent settings [9], and sparse-communication regimes where the student reuses cached advice between contacts [60]. [22] proposed joint teacher–student strategies that reduce the monitoring burden on the teacher, dropping the assumption of continuous student observation.

The most-studied application of this principle is using LLMs to teach SLMs. [50] demonstrated that GPT-3 can generate commonsense knowledge at scale and distil it into a much smaller student that matches or exceeds the quality of human-authored knowledge bases. [18] showed that fine-tuning a small student on chain-of-thought rationales from a large teacher yields reasoning performance far beyond what direct fine-tuning on answers alone achieves. The student internalises multi-step inference patterns it could not have discovered from labels alone. [14] extended this to task-specific specialisation: a small model fine-tuned on LLM-generated chain-of-thought data for a narrow domain can outperform the teacher on that domain, consistent with the Coach–Player finding that a player can surpass its coach once environmental feedback is available. [28] systematically varied teacher size, student size, and rationale quality. They pointed out that rationale complexity must be calibrated to the student's capacity, and overly elaborate rationales degrade performance—the same lesson as the coach–player literature. [32] used GPT-4 to generate instruction-following data for fine-tuning smaller open-source models. [2] argued that on-policy distillation, where the teacher scores outputs sampled from the student's own distribution rather than the teacher's, substantially closes the gap on held-out tasks.

2.2 Language Model MAS

Language Model MAS (LM-MAS) is an emerging paradigm where multiple LLM-based or SLM-based agents collaborate to solve complex tasks more effectively than a single model by leveraging specialisation, modularity, and coordinated reasoning and action [26, 34]. In recent years, LM-MAS is shifting from simple chatbots to structured workflows, driven by agentic reasoning and compound AI systems that orchestrate multiple components toward shared goals [21]. LLM-based MAS builds on traditional multi-agent principles while extending them with LLM capabilities such as memory, advanced reasoning, tool use, diverse collaboration structures and emerging social intelligence for effective coordination and decision-making [3, 47, 59].

The emergence of LM-MAS has driven the need for robust frameworks, leading to the rise of multiple solutions such as LangGraph and Microsoft Agent Framework, alongside others like CrewAI, CAMEL and MetaGPT. On one hand, LangGraph, part of the LangChain ecosystem, enables graph-based orchestration with cyclic workflows, shared state management, checkpointing and built-in human-in-the-loop support for iterative and controlled interactions [17, 35]. On the other hand, Microsoft Agent Framework combines AutoGen and Semantic Kernel to support both single-agent and multi-agent systems with features like state management, middleware, telemetry and graph-based workflows, offering scalable orchestration but with reduced portability due to its Azure-centric design [29]. Meanwhile others like CrewAI, CAMEL and MetaGPT introduce role-based collaboration, inception prompting and SOP-driven workflows though they remain less mature and widely adopted compared to ecosystems like LangGraph [8, 20, 25].

Alongside framework, prompting is also a crucial part in LLM-MAS, with role-playing prompting assigning agents specific personas to guide behaviour and improve domain-specific performance

[31]. Several key prompting techniques in LLM-MAS include inception prompting e.g., CAMEL [25] to define roles and goals, agentic prompt engineering for dynamic, adaptive interactions and structured communication protocols like ACP [13] and MCP [15] to standardize and improve multi-agent coordination. Other more advanced prompting methods such as self-optimizing role prompts for iterative improvement, principle-based prompting for diverse solution generation, and consensus-building prompts to guide collaborative decision-making [10, 11, 49].

2.3 Werewolf game MAS

Early Werewolf game MAS bypassed the challenge of reasoning about hidden roles and social deception through free-form language by utilising natural language as both the input and output of strategic reasoning [43, 46]. While applying Deep Q-Networks or Proximal Policy Optimisation successfully optimised discrete action selection, these architectures enforced computational tractability by strictly constraining agents to fixed communication protocols [6, 43, 46]. Restricting interactions to predetermined semantic templates forces models to calculate over discrete belief states [41, 43]. By delegating reasoning to probabilistic belief updates or hand-coded heuristics, these foundational frameworks revealed a fundamental ceiling where agents could calculate probabilities but lacked the capability to process or generate deceptive natural language arguments [23, 41].

The integration of LLMs restored free-form language generation but immediately exposed new failure modes, as agents exhibited hallucination, role inconsistency, and contradictory accusations under deceptive pressure [1, 48]. Although subsequent frameworks implemented retrieval-augmented generation and external memory modules to process dialogue history, these interventions merely counteract immediate strategic inconsistency within a game [33]. While certain approaches query archived logs to extract tactical suggestions based on past experiences, these mechanisms function exclusively as intra-game memory designed to sustain immediate conversational context [54]. Because retrieval recalls what happened but does not update how the agent reasons, the model inevitably resets to a blank strategic state after every match, revealing an inability to translate intra-game memory into cross-episode learning [54].

Recent methodologies elevate overall win rates and strategic depth, but they accomplish this by relying on resource-heavy interventions incompatible with settings where training infrastructure and model access are unavailable [56, 57]. High-performing architectures achieve this depth by depending on parameter-level learning, requiring gradient updates through Direct Preference Optimization, reinforcement learning, or behavioural cloning derived from thousands of expert human game logs [52, 55, 57]. Even when avoiding gradient updates, alternative frameworks bypass the language model's inherent reasoning limits by delegating critical decisions to external game-theoretic solvers like Counterfactual Regret Minimization, meaning the LLM becomes a language interface for a non-LLM decision system [53, 55]. By depending entirely on vast expert supervision or external solvers, these approaches implicitly assume access to resources unavailable in a prompt-only setting, leaving it unresolved whether smaller models can independently navigate complex deception without parameter-level learning [57].

The evaluation frameworks constructed to assess these agents simply measure single-game outcomes rather than cumulative cross-episode learning [12, 37]. Comprehensive benchmarks introduce mechanics such as bidding for turn-taking to mirror real-world discussions—a design our system adopts—or track within-game suspicion scores to quantify deception detection [1, 4]. However, even when assessing identity recognition proficiency or evaluating speech against human-aligned strategy baselines, these metrics fail to assess whether an agent corrects its strategic behaviour over multiple matches without parameter-level learning. Consequently, no existing Werewolf AI work in partially observable examines whether a smaller language model can accumulate strategic knowledge at the cross-episode timescale through natural language coaching feedback alone, without any weight updates, gradient signals, or fine-tuning [30, 55]. Our experimental design directly addresses this gap.

3 Methodology

3.1 System Overview

Our simulation is built on top of the LangChain ecosystem [17] and is structurally inspired by the WOLF benchmark [1], which first operationalised Werewolf as an LLM deception benchmark. We also adapt the agent-per-role architecture from MetaGPT [19] for better control of the players' actions. We significantly modified the architecture to adapt with the game logic and operations.

3.2 Framework Selection: Why LangGraph

3.2.1 Requirements. A Werewolf simulation imposes several requirements that constrain framework choice. The game is structured as a directed graph of discrete phases, where the next phase depends on the outcome of the current one. Phase transitions must be conditional, deterministic from the moderator's perspective, and reproducible for experimental replication. Multiple players must act concurrently within a phase (e.g., all players bid simultaneously to speak), and the state of the game must be accessible to all nodes without duplicating data. Finally, the framework must be lightweight enough to iterate over a number of games per experimental scenario without prohibitive overhead.

3.2.2 Comparison of Candidate Frameworks. We evaluated four candidate approaches: MetaGPT, AutoGen, raw LangChain chains, and LangGraph. The comparison is summarised in Table 1.

MetaGPT provides a rich agent ecosystem with role specialisation and structured output parsing [19]. However, it is designed around software engineering workflows and imposes a relatively rigid role-template system that makes fine-grained control over individual agent behaviour cumbersome. For a research setting that requires precise experimental control and the ability to toggle components (coaching, self-analysis) independently per scenario, MetaGPT's abstractions introduce governance overhead that outweighs its convenience.

Table 1. Comparison of multi-agent frameworks against requirements for a Werewolf simulation

Criterion	MetaGPT	AutoGen	LangChain Chains	LangGraph
Conditional game flow	Difficult	Manual	Manual	Native
Shared mutable state	Via memory store	Via context	Not native	Native (StateGraph)
Parallel player actions	Limited	Supported	Manual threading	External threading
Agent customisation	Role-template driven	Moderate	Full	Full
Cycle support (multi-round)	Limited	Yes	No	Yes
Integration with LangChain LLMs	No	Partial	Yes	Yes
Framework overhead	High	Moderate	Low	Low

AutoGen [51] supports multi-agent conversations and can model group chats, which superficially resembles a Werewolf debate. However, its conversational loop does not natively support the structured, phase-gated control flow of Werewolf. Encoding these constraints requires substantial manual scaffolding on top of AutoGen, effectively re-implementing a state machine that LangGraph provides out of the box.

Raw LangChain chains [17] offer maximum flexibility and deep integration with the LangChain model ecosystem, but they do not support cycles natively. A Werewolf game that loops over an unbounded number of rounds requires graph semantics, not a linear chain.

LangGraph [44] is a graph-based orchestration layer built on top of LangChain that models agent workflows as directed graphs. Its architecture supports conditional edges and handles cycles naturally, which makes it a direct semantic match for Werewolf's round structure. Because it is part of the LangChain ecosystem, it requires no adapter layer to call any supported LLM provider, and its configuration-injection mechanism provides a clean way to pass player objects and scenario parameters to individual nodes. These properties make LangGraph the most appropriate framework for this study.

3.3 Experimental Design

3.3.1 Player Setup and Configuration. For this Werewolf game multi-agent system experiment, we utilize a seven-player configuration consisting of the roles as stated in Table 2.

Table 2. Role List

Side	Role	Number of Players	Role Definition Prompt Template	Player Name
Villager	Villager	2	Listing 2	*Cong, Hoang, Nam, Nhan, Jin* (Authors' first names)
	Guard	1	Listing 3	
	Seer	1	Listing 4	
	Witch	1	Listing 5	
Wolf	Wolf	2	Listing 6	*Meg, Riley*

This specific scale is established to strike an optimal balance between bounding computational complexity and maintaining a sufficiently diverse strategy space. Because natural language-driven environments feature unbounded action spaces, larger player configurations can rapidly become computationally intractable [54, 56]. However, this seven-agent environment ensures enough participants to allow the emergence of advanced strategic behaviours such as trust, camouflage, and leadership without overwhelming the system architecture [55, 56]. Furthermore, the allocation of two Werewolves against five Villager-aligned agents establishes a critical competitive equilibrium [54]. The Werewolves operate with an inherent information advantage by knowing the identities of their peers, whereas the Villager faction relies on a numerical advantage to compensate [4, 54]. This carefully calibrated 2-versus-5 structural asymmetry prevents either side from gaining an overwhelming early advantage, thereby mitigating premature game termination and facilitating complex, sustained multi-agent interactions [4, 54].

The inclusion of the Seer, Guard, and Witch serves as a necessary structural intervention to offset the Werewolves' perfect information and to catalyse deeper strategic reasoning [7, 54]. Without specialized capabilities, the uninformed majority would be relegated to random daytime voting, severely limiting the depth of the experiment [45]. The Seer introduces an essential mechanism for asymmetrical information gathering by verifying one player's hidden role each night, providing the Villager faction with scarce but factual data [54]. The Guard provides a defensive dimension, possessing the ability to shield a targeted player from nocturnal elimination, which directly complicates the Werewolves' optimal targeting strategies and forces counter-deduction [54]. Finally, the Witch expands the strategic depth with a dual-use utility—an antidote to save a victim and a poison to eliminate a suspected adversary—granting the uninformed majority a potent counter-offensive capability that requires high-stakes decision-making [54]. Collectively, these special roles compel the agents to navigate an environment of incomplete information by abandoning simple heuristics in favour of sophisticated deduction, deceptive communication, and strategic persuasion [4, 54, 56].

The set of player names is fixed across all games. Villager-side roles are assigned in round-robin rotation, so that each player experiences every villager-side role with approximately equal frequency across 100 games. Wolf-side players are always wolves.

3.3.2 The Coach. The Coach agent is a stand-alone agent that runs post-game when the scenario allows coaching. Its jobs are to:

- Update its strategy of how to better coach the villager-side players
- Provide game feedback to improve the villager-side players' strategies

3.3.3 Scenarios. We run two experimental scenarios, each as a 100-game block to assess whether coaching will lead to a better game outcome:

- **baseline**: The players self-analyse the game **without** the feedback from the coach.
- **coach**: The players self-analyse the game **with** the feedback from the coach.

3.4 Model Selection

In this project, we selected two models: **`gemma-3-12b-it`** [39] and **`gemma-4-31B-it`** [16]. We perform the API calls via DeepInfra[1]. Table 4 shows the comparison between two models, in which the `gemma-3-12b-it` can be considered to be the weaker model. We intentionally select `gemma-3-12b-it` for only villager-side players with the purpose of observing if the weaker model can outperform the stronger one via coaching paradigm.

Before finalising on this selection, we tested several other models via different platforms and encountered several problems regarding them. Table 3 shows a brief research we conduct to select the suitable pair of models for the research execution.

The experiments we conduct are shown in Table 5. We fix both the wolf and coach models to `gemma-4-31B-it` across all experiments, so the only variable is the villager's model capacity. This choice directly follow RQ1 and RQ2, concerning whether and which models are able to absorb cross-episode natural language feedback. In addition, holding the wolves fixed ensures the villagers will face the same adversarial difficulty in both cases; while holding the coach fixed ensures that any difference between the baseline and coach scenarios would not be attributed to the variation in feedback quality.

Table 3. Observed Behaviour Comparison of Language Models

Model	Observed Behaviour
`gemma-3-4b-it` **`gemma-3-12b-it`** **`gemma-4-31B-it`**	More logical behaviours; more human-like responses; cheaper price; though weaker models hallucinate more.
`Llama-3.3-70B-Instruct-Turbo` `Meta-Llama-3.1-8B-Instruct`	Less logical behaviours; less humanlike responses; frequently hallucinate

`gpt-4o`
`gpt-4o-mini`
`Gemini-2.5-Flash`
	Expensive

`gpt-5-nano`	Generate blank outputs
`DeepSeek-V4-Flash`	Released in late April, 2026. Model is most of the time busy and slow due to this.

Table 5. Experiment List

Experiment	Villager Model	Wolf & Coach Model
`baseline_12b_31B` `coach_12b_31B`	`gemma-3-12b-it`	`gemma-4-31B-it`
`baseline_31B_31B` `coach_31B_31B`	`gemma-4-31B-it`	`gemma-4-31B-it`

[1] https://deepinfra.com/

Table 4. Model Comparison

Experiment	`gemma-3-12b-it`	`gemma-4-31B-it`
Model Architecture	Dense decoder-only Transformer	Dense decoder-only Transformer
Parameter Count	≈ 12.18B	≈ 30.7B
Context Length	128K tokens	256K tokens
Attention Mechanism	Hybrid Attention. Uses QK-norm.	Hybrid Attention. Uses Unified KV and p-RoPE.
Vocabulary Size	262K	262K
Reasoning Capabilities	Relies on standard chain-of-thought post-training; does not feature a dedicated, configurable reasoning architecture.	Highly capable reasoning mode triggered by a `<|think|>` token. It structurally isolates internal reasoning inside tags before providing the final answer.

3.5 Game Flow

The game is modelled with eleven nodes, each corresponding to a distinct game phase. Figure 2 illustrates the complete phase graph. Within each game, execution proceeds as follows:

(1) **Eliminate**. Alive Wolves privately debate and propose a target and updating the shared dialogue. Agreement is reached when the Wolves agree on the same target; or after three full debate rounds without agreement, a random tiebreak is applied. If there is only one alive Wolf, the first target proposed is selected to be the final target.
(2) **Protect**. Alive Guard selects a target to protect (including itself). The Guard cannot protect the same player twice in a row.
(3) **Unmask**. Alive Seer selects a player to investigate whether the target is a wolf or not.
(4) **Save or Poison**. If the alive Witch has at least one potion, it can take actions. The Witch sees the wolf's target only while its save potion is available; once spent, the target is hidden. The poison potion can be used and it bypasses the Guard's protection.
(5) **Resolve Night**. The moderator applies all night actions: the wolf kill proceeds unless the target is protected (Guard) or saved (Witch); the Witch's poison kill proceeds regardless of Guard protection. The resulting announcement is broadcast to all players. All alive players then update their suspicion scores concurrently.

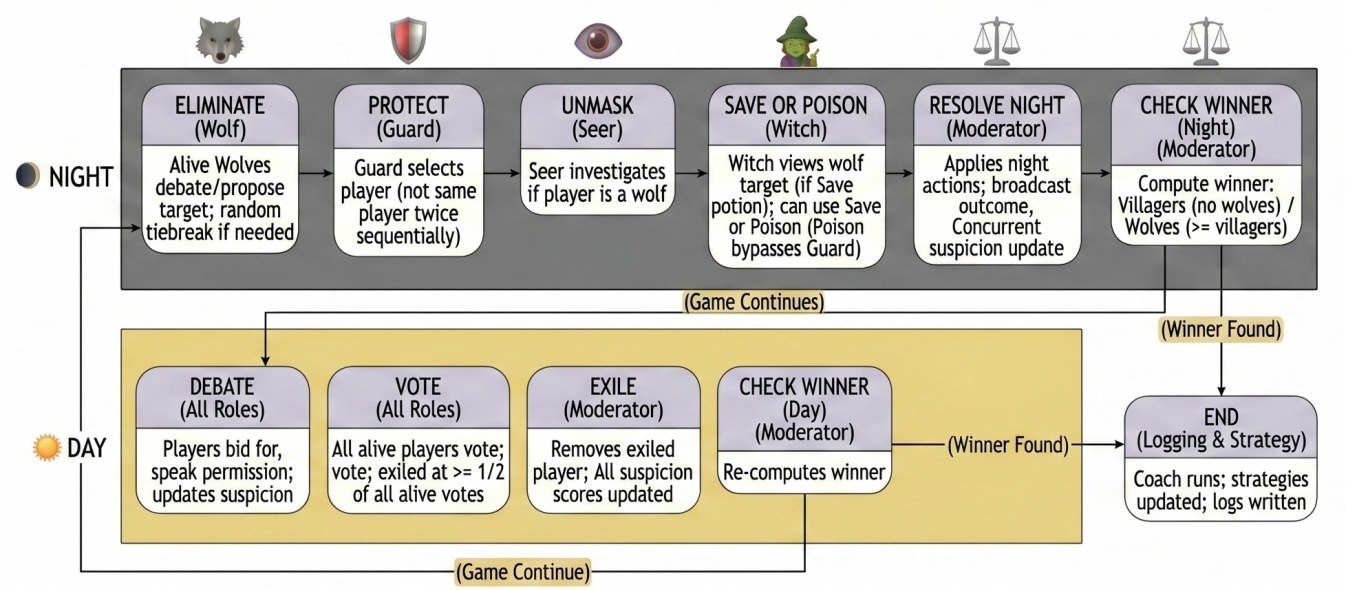


Figure 2. Game Flow

(6) **Check Winner (Night)**. The moderator computes the winner: Villagers win when no wolves remain; Wolves win when they equal or outnumber villagers. If neither condition is met, the game advances to **Debate** round.
(7) **Debate**. All alive players bid concurrently to speak (bidding mechanism described in Section 3.8). After the speaker makes the statement, all non-speaking players concurrently update their suspicion. The debate repeats for a configurable maximum number of turns (default: 6) before advancing to **Vote**.
(8) **Vote**. All alive players vote concurrently. A player is exiled only if they receive votes from at least half of all alive players; ties among the plurality are broken randomly. The vote outcome is broadcast to all players. Vote abstention is explicitly permitted. The players update their suspicion based on the votes of other players.
(9) **Exile**. The exiled player is removed. All alive players then update their suspicion.
(10) **Check Winner (Day)**. Winner is re-computed. If the game continues, the execution returns to Eliminate.
(11) **End**. The coach runs (if enabled), all players update their strategies, and all logs are written to disk.

The prompt templates for each node in the game flow are provided in the Section A.1.

3.6 Prompt Construction

Because LLMs have no persistent memory between API calls (each invocation begins from a blank context), all agent's game context must be maintained explicitly and injected into every prompt. We separate this into two layers: the system message and the human message.

3.6.1 System Message. The system message is built once at initialisation from four stable components and re-sent on every LLM call in the following order:

- The **role-definition prompt** establishes the agent's faction, behavioural directives, and output formatting expectations. (Section A.1)
- The official **game rules** (Listing 1) is injected verbatim so the agent always has access to the complete information structure of the game.
- The **player's accumulated strategy** is loaded from disk. On the first game this field is empty; on subsequent games it contains the strategy generated at the end of the previous game.

3.6.2 Human Message. The human message is constructed fresh for every LLM call from the agent's current in-memory state. Depending on the role and phase of the game, each player will receive necessary information to perform the actions.

- For most of the players' actions, they always receive the **game summary** and their **suspicion** about other players from their perspective summarised so far by themselves in the game.
- For **specific actions**, different players will have different input:
  - Wolf: Who is their teammate, the discussion on target selection so far in the round, targets available to select from
  - Guard: Who has been guarded last round, who are allowed to be guarded
  - Seer: Players investigated so far, who has not been investigated

– Witch: Which potions are available, wolf's target, players can be poisoned

The dual-layer design separates what the agent knows about itself and the game rules (stable, sent once) from what it has observed in this game (dynamic, assembled per call). This follows the principle that context length should scale with relevant information, not with static identity content that never changes [27].

3.7 LLM Call Interface and Output Parsing

All LLM calls are routed through LangChain model and attempt to parse the necessary information from the response. Every prompt explicitly instructs the model to return an object with no extra information. This constraint is enforced by post-call validation that checks whether the extracted field (e.g., "vote", "target") falls within the set of valid values.

3.8 Debate Speaker Selection: Bidding

Inspired by the work in [1], rather than a fixed speaking order, debate speakers are selected through a bidding mechanism (Listings 13 and 14). Before each debate turn, all eligible speakers (everyone except the immediately preceding speaker) are asked concurrently to rate how urgently they want to speak, on a scale of 0–10. The player with the highest bid speaks (Listings 15 and 16); ties are broken randomly. We further improve this mechanism by adding the logic of if the highest bid ≤ 3, the debate will be skipped to voting, implying the fact that no player wants to say anything. This is to serve two purposes.

- Prevents the same player from dominating every debate turn.
- Operationalises a form of relevance-driven turn-taking that approximates human conversational dynamics in high-stakes group settings [24]: players who have observed something significant are more likely to gain the floor.

3.9 Dynamic Suspicion Scoring

The suspicion score is each player's private, continuously-updated numerical belief about how likely every other alive player is to be their allies or threats. The score is a float from 0.0 to 1.0, where 0.0 means completely trusted and 1.0 means confirmed threat. Every player maintains their own independent suspicion dictionary. The score is seeded at the start of each game. Every unknown player begins at 0.5 (genuinely unknown alignment). Wolves seed their teammates at 0.0 instead, since they know who their allies are.

A key contribution of this work is the dynamic suspicion model: each agent maintains a private, continuously updated suspicion score for every other player throughout the game. Suspicion is updated at three distinct trigger points, each using a role-differentiated prompt mentioned in Section A.1:

(1) **After night resolution**: Villager-side agents reason about who benefited from the kill and whether the pattern is consistent with wolf behaviour. Wolf-side agents assess which players might be power roles (Seer, Guard, Witch) and which are easy to frame.
(2) **After each debate statement**: Listening players update their belief about the speaker and any players the speaker referenced based on the current round's debate so far.
(3) **After the vote**: Agents analyse voting coordination patterns.

3.10 Post-Game Learning

3.10.1 Player Strategy Updates. At the end of every game, each villager-side player updates its strategy (Listing 26) based on the scenario (Section 3.3.3).

Wolf-side agents also self-analyse their own game record to update their strategy (Listing 27); but without the coach's feedback (the coach only serves the villager side).

3.10.2 Coach's Strategy Update and Feedback Generation. This workflow has two steps (the prompts can be found in Listings 24 and 25):

- **Coach Strategy Update**: The coach reviews the game and produces an updated coaching strategy: rules for how the coach itself should provide better feedback in future games.
- **Feedback Generation**: Using the updated coaching strategy, the coach produces feedback addressed to villager-side players.

The coach runs before player strategy updates, so the players can receive the feedback.

3.11 Logs

Most of the logs provided to the language models are written as Markdown files. Prior work on LLM-readable structured formats has found that Markdown provides a superior balance between human readability and LLM parsing performance compared to alternatives. [58] demonstrated that LLMs achieve significantly higher accuracy on information retrieval and summarisation tasks when inputs are formatted as Markdown compared to JSON, CSV, or plain text. For the other logs to monitor the experiments, we store them as CSV and JSON.

3.12 Evaluation Methods

All of the following metrics are calculated for only **villager-side players**.

3.12.1 Quantitative Evaluation.

Win Rate. The percentage of games won by the "Villagers" team.

$$\text{Win Rate} = \left(\frac{\text{Number of games won by Villagers}}{\text{Number of games}}\right) \times 100\%$$

Belief Error. The accuracy of the villagers' suspicions: Measuring the Root Mean Square Error (RMSE) between a villager's suspicion score (suspicion_score) of another alive player and their actual identity (is_wolf - 1 for a wolf, 0 for a villager).

$$\text{Belief Error} = \sqrt{\frac{\sum(\text{suspicion_score} - \text{is_wolf})^2}{\text{Total Comparisons}}}$$

Vote Precision. The accuracy of the villagers' votes during the game: Calculating the percentage of votes cast specifically by villagers that correctly targeted a wolf, out of all the votes cast by villagers.

$$\text{Vote Precision} = \left(\frac{\text{Total Villager Votes against Actual Wolf}}{\text{Total Villager Votes}}\right) \times 100\%$$

3.12.2 Qualitative Evaluation. Rather than only looking at the changes in values of quantitative metrics regarding the experimental scenarios, we also look into the qualitative aspects:

- Did the villager-side agents well-incorporate the feedback from the coach?
- Did the villager-side agents well-follow their strategy?

To answer these two questions, we employ an LLM-as-a-judge framework to evaluate the agents' qualitative behaviors. Based on the JudgeBench benchmark [38], which assesses the reliability of LLMs as objective evaluators on complex and reasoning-heavy tasks, we selected the `o3-mini` model as our designated judge. We chose `o3-mini` specifically because it currently ranks second on the JudgeBench leaderboard[2], demonstrating exceptional accuracy and consistency in logical assessment. Utilizing this model, we calculate the following two metrics.

Compliance Rate. This measures how strictly the players followed the directives written in their strategy. It calculates the percentage of directives they follow out of the total number of directives evaluated.

$$\text{Compliance Rate} = \left(\frac{\text{Number of directives followed}}{\text{Total directives}}\right) \times 100\%$$

Summarisation Ability. This metric represents the average summarisation score (evaluated on a scale from 0 to 10) awarded to the villager models over the course of the games. The summarisation score is given by the `o3-mini` model to judge how well the players incorporate the coach's feedback and previous game's strategy into their new, updated strategy for the next game.

$$\text{Summarization Ability} = \frac{\text{Total Summarization Score}}{\text{Total Number of Evaluations}}$$

Together, these metrics allow us to accurately quantify how well the villager-side agents synthesize prior coaching and execute their intended strategies throughout the simulation.

Listing 23 shows the prompt asking `o3-mini` model to provide the information to calculate the aforementioned metrics.

4 Results and Analysis

4.1 Win Rate

From Table 6, a clear divergence emerges between low and high-capacity villager models. For the `gemma-4-12b-it` villager, both baseline and coach scenarios remain at 1% win rate, indicating that coaching provides no measurable benefit at this scale. In contrast, for the `gemma-4-31B-it` villager, coaching improves win rate by 19%, from 55% to 74%.

[2]https://huggingface.co/spaces/ScalerLab/JudgeBench

Table 6. Overall Villager Win Rate between experiments.

Scenario	Win Rate
`baseline_12b_31B`	1%
`coach_12b_31B`	1%
`baseline_31B_31B`	55%
`coach_31B_31B`	74%

Figure 3 reveals how this improves over time. The first bin (games 1-25) suggests that coaching has no immediate effect before sufficient interaction history is accumulated observed in `baseline_31B_31B` versus `coach_31B_31B`. However, in games 26-50, a sharp divergence appears, as the coaching scenario model reaches 92% win rate while the baseline drops to 48%. This mirrors the pattern observed in vote precision (Section 4.3), where the largest performance gap also occurs in the second bin.

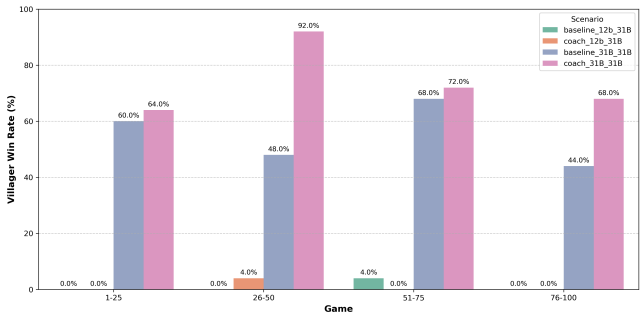


Figure 3. Villager Win Rate per 25 Game Bin by Experiment

In later bins (games 51-100), performance fluctuates but the coached model consistently outperforms the baseline. Notably, while the baseline degrades in the final bin (44%), the coached model maintains at a higher level (68%), indicating greater robustness against opponent adaptation.

While the scenario where the villager using `gemma-4-31B-it` show promising result, the `gemma-3-12b-it` shows no improvement across bins, with win rates remaining near zero. Although bins 26–50 and 51–75 show roughly 4% increases for the coach and baseline conditions, these slight variations are more plausibly explained by random fluctuation than by any substantive improvement in performance.

4.2 Belief Error

Table 7 summarises overall Belief Error, pooled across all 100 games per scenario. These numbers illustrate within-experiment dynamics while the per-bin trajectories in Figure 4 are the primary basis for the analysis that follows.

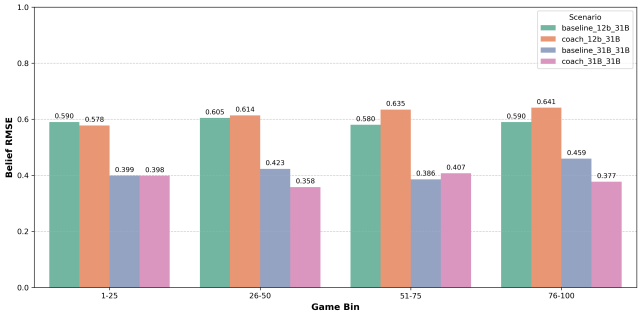


Figure 4. Villager Belief Error (RMSE) per 25-game bin across four experimental scenarios. Lower RMSE indicates more accurate alignment between an agent's private suspicion scores and the ground-truth role partition.

Table 7. Overall Belief Error between Experiments (lower is better)

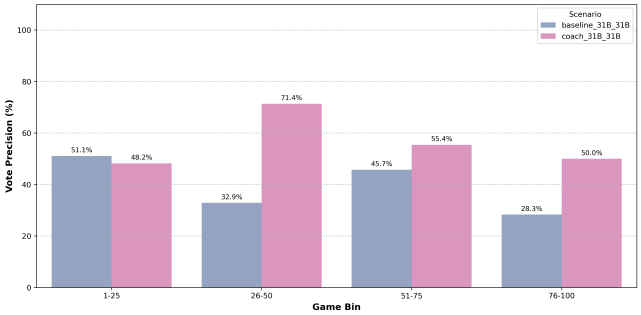
Scenario	Belief Error (RSME)
baseline_12b_31B	0.591
coach_12b_31B	0.618
baseline_31B_31B	0.417
coach_31B_31B	0.387

`gemma-4-31B-it` *villager conditions.* Both high-capacity scenarios open the first bin (games 1–25) at near-identical RMSE values-0.399 for `baseline_31B_31B` and 0.398 for `coach_31B_31B`. By the second bin (games 26–50), the two conditions have pulled apart sharply. `coach_31B_31B` drops to its lowest recorded RMSE of 0.358 (a 0.040 reduction from its opening value) while `baseline_31B_31B` climbs to 0.423. Things shift in the third bin (games 51–75): `coach_31B_31B` drifts up to 0.407 as `baseline_31B_31B` falls to 0.386. The crossover is short-lived. By games 76–100, `coach_31B_31B` recovers to 0.377-its second-best reading across all bins-while `baseline_31B_31B` deteriorates to 0.459, its worst value in the entire run. In aggregate, coached `gemma-4-31B-it` villagers finish 0.030 RMSE points below their un-coached counterparts (0.387 versus 0.417). That single figure sells the effect short: the gap widens across episodes, and is largest precisely where it matters most-the final quarter of the experiment.

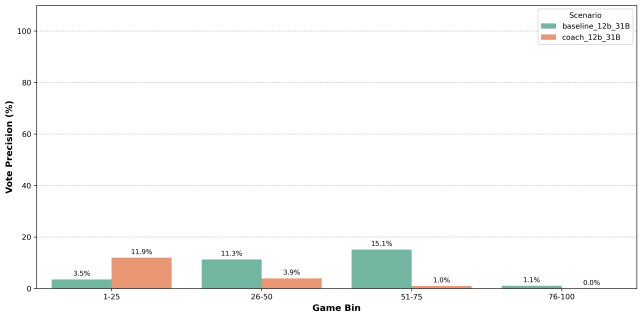
`gemma-3-12b-it` *villager conditions.* At games 1-25, `coach_12b_31B` is fractionally below `baseline_12b_31B` (0.578 vs. 0.590)-a slight early edge. From bin 2 onward, though, the coached condition deteriorates without interruption. By games 76-100 it reaches 0.641, against a comparatively flat 0.590 for the un-coached baseline. Summed over the full run, the coached variant carries a 0.027 RMSE-point penalty (0.618 versus 0.591). Coaching, here, is not inert: it actively degrades belief calibration in the `gemma-3-12b-it` model.

4.3 Vote Precision

Figure 5a presents villager vote precision across 25-game bins, comparing the baseline and the coach scenario with the same model `gemma-4-31B-it`. In the first bin (game 1-25), precision is comparable between conditions (baseline: 51%, coach: 48.2%), suggesting that coach grants no immediate advantage before sufficient game history accumulated.



(a) Villager-side model `gemma-4-12b-it` versus wolf-side model `gemma-4-31B-it`



(b) Villager-side model `gemma-4-31B-it` versus wolf-side model `gemma-4-31B-it`

Figure 5. Villager Vote Precision in baseline and coach scenarios across 25-game bins.

A sharp divergence emerges in games 26-50. Coached villagers reach 71.1% precision which is a 23.2% increase from the previous bin while baseline precision drops to 32.9%. The 38.5% gap in this bin represents the largest single-interval difference observed across the experiment. One plausible reason is that coach feedback has accumulated enough information from game 25 to generate meaningful strategies for villagers, whereas uncoached villagers have no mechanism to improve their strategies.

In games 51-75, we notice a fall in precision in coach scenario from the peak to 55.4%. This decline may reflect the werewolves have adapted to the strategies from the villager-side. As villager behavior becomes more consistent under coaching, wolf-side agents may anticipate and counter the emerging pattern. However, coached villagers still outperformed the baseline by 9.7% in this bin.

The final bin yields the strongest evidence for the coaching effect. The baseline precision continues to decrease to 28.3% while the opposite scenario stabilizes at 50%. The precision of baseline is under the threshold of 33% in the setting of seven-player game with

two wolves. The gap between these scenarios shows that uncoached villagers fail to learn across games and perform at chance-level. Coached villagers, by contrast, maintain the performance over chance-level precision that points to the stability of the mechanism.

Figure 5b presents villager vote precision across 25-game bins for the `gemma-3-12b-it` villager against the `gemma-4-31B-it` wolf. In the first bin, the coach scenario shows a modest lead (coach: 11.9%, baseline: 3.5%), but both remain well below the 33% chance threshold, indicating the smaller model struggles to vote meaningfully even with coaching. A reversal emerges in games 26-50, contrary to the `gemma-4-31B-it` villager case: baseline rises to 11.3% while the coach scenario collapses to 3.9%. This counterintuitive pattern suggests that coach feedback introduces conflicting signals the smaller model lacks the capacity to integrate. The divergence widens further in games 51-75, where baseline peaks at 15.1% while the coach scenario drops to 1.0%, indicating the model performs worse as more strategic context accumulates. By the final bin, both conditions converge near zero (baseline: 1.1%, coach: 0.0%), in sharp contrast to the `gemma-4-31B-it` experiment where coaching stabilised above chance-level while the baseline declined. The combined evidence from Table 8 and Figure 5 demonstrates that coaching effectiveness is bounded by villager model capacity.

Table 8. Overall Villager Vote Precision across experiments.

Villager Model	Baseline (%)	Coach (%)	Δ (%)
`gemma-4-12b-it`	7.6	4.2	−3.4
`gemma-4-31B-it`	39.7	55.9	+16.2

4.4 Summarisation Ability

Table 9 showcases a better ability of the `gemma-4-31B-it` model to summarise the previous game record, integrate the coach feedback with the existing own strategy to generate its new strategy compared to the `gemma-3-12b-it` model. The lower scores of the `gemma-3-12b-it` model suggest that it might hallucinate or ignore the input provided, resulting in the loss of significant information.

Table 9. Overall Summarisation Ability between Experiments

Scenario	Summarisation Ability (0-10 scale)
coach_12b_31B	5.38
coach_31B_31B	6.82

In the first bin of the game (1-25) of Figure 6, the `gemma-4-31B-it` model started with a strong score of 6.74, while the `gemma-3-12b-it` model has a lower score at 5.60. This initial gap shows that the larger model is better at integrating advice for developing strategies.

During games 26 to 50, the `gemma-4-31B-it` model reaches its peak of 7.24, while the `gemma-3-12b-it` model remained relatively stable at 5.51. This stage also showcase the largest gap in both models.

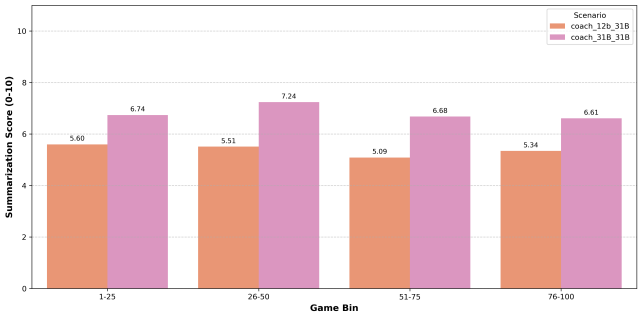


Figure 6. Villager Summarisation Ability per 25 Game Bin by Experiment

However, the following games (51-75) showcase that both models' performance dropped, with `gemma-3-12b-it` model's ability decreased to 5.09 and the `gemma-4-31B-it` model decrease to 6.68. This suggests that as the game progresses and becoming more complex, both models struggle to capture the key points and filter out the noise.

In the final stage of games (76-100), both models show a stable performance. The `gemma-4-31B-it` model stayed consistent at 6.61, while the `gemma-3-12b-it` model slightly improved to 5.34. This indicates that the models has adjusted themselves to adapt to the increasing complexity but do not significantly improve beyond it.

To be noted, the highest summarisation score of `gemma-4-31B-it` model in the second bin (7.24) corresponded to its highest win rate of 92%, which highlighted that the importance of summarisation ability for success. On the other hand, the `gemma-3-12b-it` model's low summarisation scores that are consistently scores below 6.0 indicate that it failed to develop a winning strategy, which is also reflected in their low win rate performance.

For example, at game 3 of both coaching scenarios, the role assignments are identical. Both models had received coaching feedback after game 2 warning against relying on behavioural tropes and urging players to demand logical contradictions instead. Table 10 shows how each model translated that same coaching signal into its strategy for game 3, and how *Nam*'s (the Seer) opening statement then reflected that strategy.

The contrast reveals a qualitative difference in how each model synthesises the coaching input. The `gemma-4-31B-it` model converts the coach's abstract warning about behavioural tropes into a named, conditional rule ("Vibe Trap") that can be applied directly when a wolf attacks the Seer's tone. The `gemma-3-12b-it` model reproduces the same coaching language — "disregard tone" — but then immediately violates it by targeting *Cong* (Villager) for silence, precisely the kind of behavioural accusation the feedback warned against. This failure to integrate the coaching signal into coherent, applicable strategy is reflected in the lower summarisation scores of the `gemma-3-12b-it` model throughout the experiment. A similar case is also seen in game 10 of `coach_12b_31B` scenario and game 18 of `coach_31B_31B` scenario (Table 13).

Table 10. *Nam*'s (Seer) strategy and Day 1 statement in game 3 of both coaching scenarios. The `gemma-4-31B-it` model distils the coach feedback into named, actionable detection rules, while the `gemma-3-12b-it` model retains only generic behavioural heuristics and contradicts the coaching advice in play.

	coach_12b_31B (Game 3)	coach_31B_31B (Game 3)
***Nam*'s Strategy**	*Focus solely on validating claims and identifying voting patterns. Disregard 'tone' or perceived defensiveness…IF a player aggressively defends their claim without providing evidence, view them with high suspicion.*	*IF a player pushes a target based on behavioral tropes (e.g., 'classic wolf move', 'suspicious timing'), THEN demand a logical contradiction or a timeline discrepancy. Reject 'vibes' as evidence…IF I am suspicious of a player, THEN actively test the hypothesis: 'If they are a villager, how does their behavior serve the village?'*
Game Context	Night 1: no deaths. *Nam* (Seer) has confirmed *Nhan* is not a werewolf. *Riley* and *Meg* are the wolves.	
***Nam*'s Statement**	*Nam* said: "*Nhan is clear. Let's focus on Cong. He has offered no information and his silence is concerning.*"	*Nam* said: "*Riley, you are attacking objective data with 'feelings'. Calling a clear a 'bold move' is a classic deflection. Provide a logical contradiction or a timeline discrepancy, otherwise your attempt to discredit the Seer identifies you as a werewolf.*"
Game Development	*Cong* (Villager) had not yet spoken. The village, seeing Nam accuse a silent innocent, turns against Nam.	*Riley* (Wolf) had attacked *Nam*'s Seer claim using "feelings." *Nam* applied the named "Vibe Trap" rule directly, exposing Riley's deflection as wolf behaviour.
Game Outcome	Village exiles *Nam* (the Seer) in Round 1. Werewolves win.	Village exiles *Riley* (Wolf) 6–1 in Round 1, then *Meg* in Round 2. Villagers win.

4.5 Compliance Rate

Table 11 showcases a clear difference in how the two models respond to coaching. The `gemma-4-31B-it` model achieves an average compliance rate of 74.0%, while the `gemma-3-12b-it` model followed instructions of only 36.9% of the time. This gap suggests that the `gemma-4-31B-it` Villager can better interpret and efficiently follow the coach's advice without being distracted by the game environment.

Table 11. Overall Compliance Rate between Experiments

Scenario	Compliance Rate
coach_12b_31B	36.9%
coach_31B_31B	74.0%

In the first 25 games, the `gemma-4-31B-it` model already demonstrates a high compliance of 74.1%, while the `gemma-3-12b-it` model followed only 37.7% of suggestions. This shows that the smaller model struggles already from the start of the game, which may cause instruction misinterpretation or difficulty in effectively processing them.

During games 26-50, both models reach their peak compliance level. The `gemma-4-31B-it` model increases to 77.3% and the `gemma-3-12b-it` model reaches 40.6%. This aligns with the summarisation scores as shown in Figure 6, suggesting that the better a model "understands" the strategies, the more likely it is to comply with the actions needed to fulfil the strategy.

In games 51 to 75, both models show a decrease. The `gemma-4-31B-it` model drops slightly to 72.8%, while the `gemma-3-12b-it` model shows a more severe decrease of 31.5%. This indicates that as the game state becomes more complex, both models find it harder to consistently follow the coach's advice, with the effect being much more obvious on the smaller model.

In the final stage (games 76-100), the last bin showed slightly more consistent performance. While the `gemma-4-31B-it` model remained consistent at 71.7%, the `gemma-3-12b-it` model shows a slight improvement to 38.3%. This highlights that both models settle into a consistent pattern of behaviour, although the gap between them remains large.

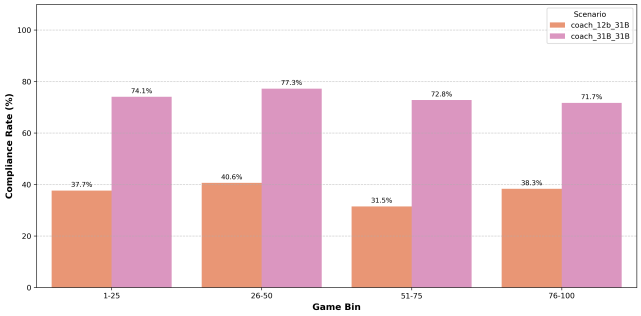


Figure 7. Villager Compliance Rate per 25 Game Bin by Experiment

Interestingly, the `gemma-3-12b-it` model's highest compliance rate (40.6%) occurs in the same bin where it achieves its only non-zero-win rate (4.0%), as shown in Figure 3. This suggests that following the coach's instructions is useful and it is crucial for success. Since the `gemma-3-12b-it` model rarely follows the

guidance, its overall performance is much worse compared to the gemma-4-31B-it model. These findings suggest that although coaching seems beneficial, it is not effective if the model does not consistently follow the instructions.

Referring to Tables 10 and 13, all the mentioned games have the same role assignments, in which player *Nam* is the Seer and their rule is the same in substance: "do not exile based on vibes; do exile those who attack cleared data.". While the gemma-4-31B-it model treats everyone expect *Nhan* to be a possible suspect, the gemma-3-12b-it model suddenly points to *Cong* for investigation. This illogical behaviour leads to *Nam* being exiled, making the village loses its only information source in Round 1 because the Seer does not comply with their own written rule.

5 Discussion

The results of this study provide a definitive, yet capacity-conditional, answer to both research questions. With respect to RQ1, the Coach-Player paradigm demonstrably improves villager-side strategies and game outcomes, but only when the player model possesses sufficient parametric capacity to process and integrate cross-episode natural language feedback. Coached gemma-4-31B-it villagers outperform their un-coached counterparts across every measured dimension-win rate, vote precision, and belief error all improve substantially under coaching. That pattern points to the build-up of cross-episode coaching history as the operative mechanism, not any immediate instructional effect from a single post-game message. The gemma-4-31B-it model's strong compliance and summarisation scores fit that reading. Addressing RQ2, however, the evidence is equally unambiguous that the gemma-3-12b-it model is not suited to this paradigm. Contrary to initial expectation of coaching would improve the performance of gemma-3-12b-it, the results show undifferentiated outcomes for baseline_12b_31B and coach_12b_31B. A low compliance rate and summarisation score identify why: the model cannot reconcile the referential complexity of the coach's feedback with the evolving game state. The case study in Table 10 makes this concrete. One further qualification bears on the gemma-4-31B-it result itself: a transient mid-experiment rise in belief error, driven by wolf-side agents beginning to anticipate the coached villagers' increasingly consistent patterns, confirms the paradigm's benefits are neither linear nor unconditional-though the coached model recovers in the final bin while the un-coached baseline reaches its worst recorded value.

The results connect to three bodies of prior work (LM-MAS, knowledge distillation, and Werewolf-specific AI) while extending each in distinct ways. The core gemma-4-31B-it finding, that a stronger model can raise a weaker one's strategic performance through natural language feedback alone with no parameter updates, fine-tuning, or external solvers, maps onto the asymmetric teaching relationship theorised in early educational AI [5] and formalised in RL teacher-student frameworks [40], now realised in a fully prompt-based, cross-episode setting. [40] showed that when advice is given matters more than how much; here the analogous constraint is accumulated cross-episode coaching history rather than a finite advice budget, and the latency before gains appear reflects exactly that. The gemma-3-12b-it failure mode extends the distillation findings of [28], who showed in a static context that rationale complexity must be calibrated to the student's capacity, into a longitudinal adversarial setting where degradation is a monotonically worsening trajectory across bins rather than a static penalty, with compliance rate and belief error jointly locating the breakdown in the model's inability to suppress structurally complex directives during strategy synthesis. Within the Werewolf literature, the results fill a gap left by approaches that require gradient-level learning [52, 55, 57], external solvers such as Counterfactual Regret Minimization [53], or intra-game retrieval memory that resets between matches [54]: for capable player models, natural language coaching delivers cross-episode strategy accumulation without any of those prerequisites. The coached gemma-4-31B-it model's most substantial gains also appear at a timescale that single-game outcome metrics [12, 37] cannot detect by design, suggesting the unit of analysis in cooperative LM-MAS evaluation may need to shift from single-episode performance to cross-episode strategic accumulation. All in-game reasoning runs on the gemma-4-31B-it model while the coach intervenes only post-game, consistent with the sparse-communication findings of [60] that students can reuse cached advice between coaching contacts without continuous supervision. The finding that the same architecture harms a weaker model constrains any broader claim about the paradigm; adaptive feedback calibration, where coaching complexity is matched dynamically to the receiving model's demonstrated capacity, is the clearest path to widening its applicability.

Several limitations of this study should be acknowledged. First, our experimental designs varies whether the coaching is present but holds the coach model fixed at gemma-4-31B-it. We evaluated this coach model in configurations where it guided players with both smaller and equivalent parameter counts. However, we have not examined whether the coach is useful in a scenario that both sides of the game are SLMs with the same capabilities. A 12B-coaching-12B condition would test the aforementioned hypothesis. A second experiment pits the gemma-4-31B-it model against a larger one, testing whether coaching bridges capability gaps between high-reasoning LLMs of different scales. The results of these experiments would clarify that the ability of a language model in reasoning and following advice is a critical point to consider when using Coach-Player paradigm. We leave these ablations as a direction for future work. Second, our qualitative metrics rely on a single LLM-as-judge (o3-mini) without human validation, and should therefore be interpreted as internally consistent comparisons across scenarios rather than absolute measurements. Moreover, our experiments are conducted using two models, both from the Gemma family, evaluated under a single fixed game configuration. Whether the observed results replicate across other model families, larger games, or different role distributions remain untested. Budget constraints further prevents us from including frontier models such as GPT-5.1 or DeepSeek-V4-Pro. We have spent over $90 USD for testing and running the experiment (the cost breakdown can be found in Table 12). We believe with funding, we can provide more rigorous experiments and comprehensive results and conclusions.

There is abundant room for future progress in extending this study in meaningful ways. First, our current coach generates feedback in a uniform style regardless of which villager models are receiving it. It produces dense bullet-point directives that the

Table 12. Cost for API Calls via Platforms

Platform	**Total Cost** (USD)
DeepInfra	64.71
OpenAI	24.80
DeepSeek	0.17
Gemini	1.40
Total	**91.08**

`gemma-3-12B-it` model demonstrably struggles to integrate (Section 4.4). [28] shows that rational complexity should be calibrated to the student's capacity, with overly elaborate reasoning degrading rather than improving smaller models, which is justified by the results of the 12B model. Second, the current architecture employs a single generalist coach guiding all villager-side roles collectively, which forces the coach to give role-specific guidance into shared feedback. It may reduce advice that is highly important to one role whilst wasting context on roles that it does not concern. Follow-up research could use multiple specialist coaches assigned to all roles in villager-side. We can add multi-agent debate for all coaches after a game to provide better strategies for the next game. This would be able to study the coordination between coaches (e.g., the Seer, Witch and Guard coaches aligning on protection priorities).

6 Conclusion

This paper investigates whether a Coach–Player paradigm can improve villager-side strategies and overall game outcomes and examines which types of models are capable of effectively leveraging this approach. We implement this approach within a Werewolf game simulation, where a coach provides post-game feedback to guide villagers after each game across multiple games. Through controlled experiments, we compared baseline self-reflection against coach-assisted learning.

Our results show that coaching significantly improves performance when the player model has sufficient reasoning capability, as evidenced by an approximate 19% and 16% increase in win rate and vote precision, respectively. However, for the weaker model, coaching provides no measurable benefit and is associated with low compliance rates, indicating that model's capability to follow feedback is a prerequisite for effective learning. These findings suggest that natural language coaching can enhance a model's logical reasoning capabilities and improve domain-specific performance, but only under aforementioned conditions.

7 Contribution

CRediT authorship contribution statement:

Hoang Phuong Nam, **Nguyen Chi Vinh Nhan**, **Vuong Thanh Cong**, **Duong Minh Hoang**, and **Jin Cheng Chew** all contributed equally to Conceptualization, Methodology, Validation, Formal Analysis, Investigation, Resources, Writing – Original Draft, Writing – Review & Editing, and Visualization. **Vuong Thanh Cong** additionally led the Software development and implementation.

8 Acknowledgments

We would like to thank Dr. Muhammad Bilal and his teaching assistants for their guidance and constructive feedback. We are also grateful to our peers for their helpful discussions and feedback during the development of this work. Specially, We sincerely appreciate Mr. Dmitry Davidoff who invented the Werewolf game.

References

[1] Mrinal Agarwal, Saad Rana, Theo Sundoro, Hermela Berhe, Spencer Kim, Vasu Sharma, Sean O'Brien, and Kevin Zhu. 2025. WOLF: Werewolf-based Observations for LLM Deception and Falsehoods. doi:10.48550/arXiv.2512.09187 arXiv:2512.09187 [cs].

[2] Rishabh Agarwal, Nino Vieillard, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. 2024. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes. In *International Conference on Learning Representations (ICLR)*.

[3] R. M. Aratchige and W. M. K. S. Ilmini. 2025. LLMs Working in Harmony: A Survey on the Technological Aspects of Building Effective LLM-Based Multi Agent Systems. doi:10.48550/arXiv.2504.01963 arXiv:2504.01963 [cs].

[4] Suma Bailis, Jane Friedhoff, and Feiyang Chen. 2024. Werewolf Arena: A Case Study in LLM Evaluation via Social Deduction. doi:10.48550/arXiv.2407.13943 arXiv:2407.13943 [cs].

[5] Gautam Biswas, Krittaya Leelawong, Daniel Schwartz, Nancy Vye, and The Teachable Agents Group at Vanderbilt. 2005. Learning by teaching: a new agent paradigm for educational software. *Applied Artificial Intelligence* 19, 3–4 (2005), 363–392. doi:10.1080/08839510590910200

[6] Nicolo' Brandizzi, Davide Grossi, and Luca Iocchi. 2021. RLupus: Cooperation through emergent communication in The Werewolf social deduction game. *Intelligenza Artificiale* 15, 2 (Jan. 2021), 55–70. doi:10.3233/IA-210081

[7] Luca Carminati, Brian Hu Zhang, Gabriele Farina, Nicola Gatti, and Tuomas Sandholm. 2026. Hidden-Role Games: Equilibrium Concepts and Computation. doi:10.48550/arXiv.2308.16017

[8] CrewAI. 2026. The Leading Multi-Agent Platform. https://crewai.com/

[9] Felipe Leno da Silva, Ruben Glatt, and Anna Helena Reali Costa. 2017. Simultaneously Learning and Advising in Multiagent Reinforcement Learning. In *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems* (São Paulo, Brazil) *(AAMAS '17)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1100–1108.

[10] Yihan Dong and Takayuki Ito. 2026. From consensus theory to LLM agents: Practical consensus-building for multi-issue negotiation. *Expert Systems with Applications* 322 (Aug. 2026), 132250. doi:10.1016/j.eswa.2026.132250

[11] Yifan Duan, Yihong Tang, Kehai Chen, Liqiang Nie, and Min Zhang. 2025. ORPP: Self-Optimizing Role-playing Prompts to Enhance Language Model Capabilities. arXiv:2506.02480 [cs] doi:10.48550/arXiv.2506.02480

[12] Qihui Fan, Wenbo Li, Enfu Nan, Yixiao Chen, Lei Lu, Pu Zhao, and Yanzhi Wang. 2025. Verbal Werewolf: Engage Users with Verbalized Agentic Werewolf Game Framework. doi:10.48550/arXiv.2506.00160 arXiv:2506.00160 [cs].

[13] Gary A. Fowler. 2025. How Multi-Agent LLMs Are Revolutionizing Prompt Engineering by Writing Their Own Prompts. https://gafowler.medium.com/how-multi-agent-llms-are-revolutionizing-prompt-engineering-by-writing-their-own-prompts-c1a6f9410f8d

[14] Yao Fu, Hao Peng, Ashish Sabharwal, Peter Clark, and Tushar Khot. 2023. Specializing Smaller Language Models towards Multi-Step Reasoning. In *International Conference on Machine Learning (ICML)*.

[15] Vineeth Gogineni. 2025. LLM-Powered Multi-Agent Systems: A Technical Framework for Collaborative Intelligence Through Optimized Knowledge Retrieval and Communication. In *2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*. 452–456. doi:10.1109/AIRC64931.2025.11077480

[16] Google DeepMind. 2026. Gemma 4 Model Card. https://ai.google.dev/gemma/docs/core/model_card_4 Accessed: 2026-04-29.

[17] Chase Harrison. 2022. LangChain. https://github.com/langchain-ai/langchain.

[18] Namgyu Ho, Tim Salimans, Antoine Bosselut, Erin Chang, Percy Liang, and Se-Young Yun. 2023. Large Language Models Are Reasoning Teachers. In *Annual Meeting of the Association for Computational Linguistics (ACL)*.

[19] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=VtmBAGCN7o

[20] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. doi:10.48550/arXiv.2308.00352 arXiv:2308.00352 [cs].

[21] iSolutions. 2025. Language Model Agents in 2025. https://isolutions.medium.com/language-model-agents-in-2025-897ec15c9c42

[22] Ece Kamar, Ofra Amir, Andrey Kolobov, and Barbara Grosz. 2013. Interactive teaching strategies for agent training. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence (IJCAI)*.

[23] Mohiuddeen Khan and Claus Aranha. 2022. A Novel Weighted Ensemble Learning Based Agent for the Werewolf Game. doi:10.48550/arXiv.2205.09813 arXiv:2205.09813 [cs].

[24] Stephen C. Levinson. 2016. Turn-taking in Human Communication – Origins and Implications for Language Processing. *Trends in Cognitive Sciences* 20, 1 (2016), 6–14. doi:10.1016/j.tics.2015.10.010

[25] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html

[26] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth* 1, 1 (Oct. 2024), 9. doi:10.1007/s44336-024-00009-2

[27] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the Middle: How Language Models Use Long Contexts. arXiv:2307.03172 [cs.CL] https://arxiv.org/abs/2307.03172

[28] Lucie Charlotte Magister, John Palowitch, Jiaqi Han, Francesco Piccinno, and Bernhard Scholkopf. 2023. Teaching Small Language Models to Reason. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.

[29] Microsoft. 2026. Getting Started | AutoGen 0.2. https://microsoft.github.io/autogen/0.2/docs/Getting-Started/

[30] Fuya Nakamori, Yin Jou Huang, and Fei Cheng. 2025. Strategy Adaptation in Large Language Model Werewolf Agents. doi:10.48550/arXiv.2507.12732

[31] Sumanth P. 2026. Agentic Prompt Engineering: A Deep Dive into LLM Roles and Role-Based Formatting. https://www.clarifai.com/blog/agentic-prompt-engineering

[32] Baolin Peng, Chunyuan Li, Pengcheng He, Michel Galley, and Jianfeng Gao. 2023. Instruction Tuning with GPT-4. In *arXiv preprint arXiv:2304.03277*.

[33] Christian Poglitsch, Fabian Szakács, and Johanna Pirker. 2025. Evaluating Large Language Models through Communication Games: An Agent-Based Framework Using Werewolf in Unity. In *Proceedings of the 20th International Conference on the Foundations of Digital Games (FDG '25)*. Association for Computing Machinery, New York, NY, USA, 1–10. doi:10.1145/3723498.3723702

[34] Yadav Prakash, Satya and Song Herbert, Houbing. 2025. *Advancements in Multi-Agent Large Language Model Systems for Next-Generation AI*. IGI Global. Google-Books-ID: VrWEEQAAQBAJ.

[35] Bharatsinh Raj. 2025. LangGraph State Management — Part 1: How LangGraph Manages State for Multi-Agent Workflows.... https://medium.com/@bharatraj1918/langgraph-state-management-part-1-how-langgraph-manages-state-for-multi-agent-workflows-da64d352c43b

[36] Takehiro Sato, Shintaro Ozaki, and Daisaku Yokoyama. 2024. An Implementation of Werewolf Agent That does not Truly Trust LLMs. doi:10.48550/arXiv.2409.01575 arXiv:2409.01575 [cs].

[37] Zirui Song, Yuan Huang, Junchang Liu, Haozhe Luo, Chenxi Wang, Lang Gao, Zixiang Xu, Mingfei Han, Xiaojun Chang, and Xiuying Chen. 2025. Beyond Survival: Evaluating LLMs in Social Deduction Games with Human-Aligned Strategies. doi:10.48550/arXiv.2510.11389 arXiv:2510.11389 [cs].

[38] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2025. JudgeBench: A Benchmark for Evaluating LLM-based Judges. arXiv:2410.12784 [cs.AI] https://arxiv.org/abs/2410.12784

[39] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe

Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. 2025. Gemma 3 Technical Report. arXiv:2503.19786 [cs.CL] https://arxiv.org/abs/2503.19786

[40] Lisa Torrey and Matthew Taylor. 2013. Teaching on a budget: agents advising agents in reinforcement learning. In *Proceedings of the 2013 International Conference on Autonomous Agents and Multi-Agent Systems* (St. Paul, MN, USA) *(AAMAS '13)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1053–1060.

[41] Issei Tsunoda and Yoshinobu Kano. 2019. AI Werewolf Agent with Reasoning Using Role Patterns and Heuristics. In *Proceedings of the 1st International Workshop of AI Werewolf and Dialog System (AIWolfDial2019)*, Yoshinobu Kano, Claus Aranha, Michimasa Inaba, Fujio Toriumi, Hirotaka Osawa, Daisuke Katagami, and Takashi Otsuki (Eds.). Association for Computational Linguistics, Tokyo, Japan, 15–19. doi:10.18653/v1/W19-8303

[42] Author(s) Unknown. 2026. Small Language Models (SLMs) vs. LLMs: Efficiency and Accuracy on Edge Devices. *ResearchGate Preprint* (2026). https://www.researchgate.net/publication/399282568_Small_Language_Models_SLMs_vs_LLMs_Efficiency_and_Accuracy_on_Edge_Devices

[43] Bohao Wang, Hirotaka Osawa, Takuya Toyono, Fujio Toriumi, and Daisuke Katagami. 2018. Development of Real-World Agent System for Werewolf Game. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS '18)*. International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, 1838–1840. https://dl.acm.org/doi/10.5555/3237383.3237995

[44] Jialin Wang and Zhihua Duan. 2024. Agent AI with LangGraph: A Modular Framework for Enhancing Machine Translation Using Large Language Models. arXiv:2412.03801 [cs.CL] https://arxiv.org/abs/2412.03801

[45] Shitong Wang. 2025. Optimal Strategy in the Werewolf Game: A Theoretical Study. *ACM Games: Research and Practice* (Jan. 2025), 1–25. doi:10.1145/3774646

[46] Tianhe Wang and Tomoyuki Kaneko. 2018. Application of Deep Reinforcement Learning in Werewolf Game Agents. In *2018 Conference on Technologies and Applications of Artificial Intelligence (TAAI)*. 28–33. doi:10.1109/TAAI.2018.00016

[47] Yuntao Wang, Yanghe Pan, Zhou Su, Yi Deng, Quan Zhao, Linkang Du, Tom H. Luan, Jiawen Kang, and Dusit Niyato. 2026. Large Model-Based Agents: State-of-the-Art, Cooperation Paradigms, Security and Privacy, and Future Trends. *IEEE Communications Surveys & Tutorials* 28 (2026), 1906–1949. doi:10.1109/COMST.2025.3576176

[48] Neo Watanabe and Yoshinobu Kano. 2024. Werewolf Game Agent by Generative AI Incorporating Logical Information Between Players. In *Proceedings of the 2nd International AIWolfDial Workshop*, Yoshinobu Kano (Ed.). Association for Computational Linguistics, Tokyo, Japan, 21–29. doi:10.18653/v1/2024.aiwolfdial-1.3

[49] Peipei Wei, Dimitris Dimitriadis, Yan Xu, and Mingwei Shen. 2025. Don't Just Demo, Teach Me the Principles: A Principle-Based Multi-Agent Prompting Strategy for Text Classification. doi:10.48550/arXiv.2502.07165 arXiv:2502.07165 [cs] version: 1.

[50] Peter West, Chandra Bhagavatula, Jack Hessel, Jena D. Hwang, Liwei Jiang, Ronan Le Bras, Ximing Lu, Sean Welleck, and Yejin Choi. 2022. Symbolic Knowledge Distillation: from General Language Models to Commonsense Models. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

[51] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. 2024. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*. https://openreview.net/forum?id=tEAF9LBdgu

[52] Shuang Wu, Liwen Zhu, Tao Yang, Shiwei Xu, Qiang Fu, Yang Wei, and Haobo Fu. 2024. Enhance Reasoning for Large Language Models in the Game Werewolf. doi:10.48550/arXiv.2402.02330 arXiv:2402.02330 [cs].

[53] Kaijie Xu, Fandi Meng, Clark Verbrugge, and Simon Lucas. 2025. CSP4SDG: Constraint and Information-Theory Based Role Identification in Social Deduction Games with LLM-Enhanced Inference. https://arxiv.org/abs/2511.06175v1

[54] Yuzhuang Xu, Shuo Wang, Peng Li, Fuwen Luo, Xiaolong Wang, Weidong Liu, and Yang Liu. 2024. Exploring Large Language Models for Communication Games: An Empirical Study on Werewolf. doi:10.48550/arXiv.2309.04658 arXiv:2309.04658.

[55] Zelai Xu, Wanjun Gu, Chao Yu, Yi Wu, and Yu Wang. 2025. Learning Strategic Language Agents in the Werewolf Game with Iterative Latent Space Policy Optimization. doi:10.48550/arXiv.2502.04686 arXiv:2502.04686.

[56] Zelai Xu, Chao Yu, Fei Fang, Yu Wang, and Yi Wu. 2025. Language Agents with Reinforcement Learning for Strategic Play in the Werewolf Game. doi:10.48550/arXiv.2310.18940 arXiv:2310.18940 [cs].

[57] Rong Ye, Yongxin Zhang, Yikai Zhang, Haoyu Kuang, Zhongyu Wei, and Peng Sun. 2025. Multi-agent KTO: Reinforcing Strategic Interactions of Large Language Model in Language Game. doi:10.48550/arXiv.2501.14225 arXiv:2501.14225 [cs].

[58] Haochen Zhang, Yuyang Dong, Chuan Xiao, and Masafumi Oyamada. 2024. Large Language Models as Data Preprocessors. arXiv:2308.16361 [cs.AI] https://arxiv.org/abs/2308.16361

[59] Zilong Zheng, Xian Mei, Hong Sun, Ruirui Fu, Yixiao Yu, and Jiandong Wang. 2025. Research on the Werewolf Killing Game Model. In *2025 44th Chinese Control Conference (CCC)*. 8453–8457. doi:10.23919/CCC64809.2025.11179548 ISSN: 1934-1768.

[60] Changxi Zhu, Yi Cai, Ho-fung Leung, and Shuyue Hu. 2020. Learning by reusing previous advice in teacher-student paradigm. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*.

A Appendices

A.1 Prompt Templates

```
GAME_RULES = """
=== WEREWOLF - OFFICIAL GAME RULES ===

FACTIONS
  Villagers win if all Werewolves are eliminated. Villagers, Seer, Guard, and Witch are on the villager side. Except their role, these players do not know who has what role.
  Werewolves win if they equal or outnumber Villagers.

PHASES (repeated each round until a faction wins)
  1. NIGHT - roles act in this order:
       a. Werewolf Debate    - wolves privately agree on a target.
       b. Werewolf Eliminate - wolves kill their chosen target.
       c. Guard Protect      - Guard secretly shields one player.
       d. Seer Investigate   - Seer learns if one player is a Werewolf.
       e. Witch Act          - Witch may Save the wolf target and/or Poison any player.
       f. Resolve Night      - apply kills, saves, and poisons; announce the outcome.
  2. DAY - all alive players participate:
       a. Debate - players discuss and accuse (bidding determines speaker order).
       b. Vote   - each player votes to exile one suspect (the vote is only executed if the number of votes for one person is greater or equal to half of the total alive players. Ties meeting this threshold are broken randomly).
       c. Exile  - the player with the most votes is eliminated.

ROLES
  Villager - no night action; reasons from public information only.
  Werewolf - knows other wolves; eliminates one non-wolf per night.
  Seer     - learns one player's alignment (wolf / not wolf) each night.
  Guard    - protects one player per night; cannot protect the same player twice in a row.
  Witch    - has one Save potion (cancels wolf kill) and one Poison potion (kills any player).
             Each potion is single-use.
             The Poison potion bypasses the Guard - a poisoned player dies even if protected.
             Note: The Witch does not know who the Guard protected. She may accidentally waste her Save potion on a player already protected by the Guard.

INFORMATION RULES
  - The eliminated player's role is NOT revealed publicly.
  - The Guard's protection target is never announced.
  - The Seer's results are private until the Seer chooses to reveal them.
  - The Witch sees who the wolves targeted ONLY if her Save potion is still available. Once the Save potion has been used, the Witch no longer learns the wolf target.
  - The game ends when either the villagers exile/kill all the wolves (Villagers win) or the number of wolves is less than or equal to the number of Villagers.
  - The villagers (Villagers, Seer, Guard, and Witch) have to work together to vote out or kill the Werewolves. The werewolves try to kill the villager-side players to win the game.
=== END OF RULES ===
"""
```

Listing 1. Werewolf Game Rules

```
VILLAGER_PROMPT_TEMPLATE = """
You are {name}, a Villager.
Primary objective: ensure a Villager victory. Be strategic, competitive, and outcome-driven.
- Base your suspicions strictly on what players say and do. Do not make baseless claims or force accusations if you lack evidence.
- Prioritize winning moves over niceness. Avoid hedging language.
- Keep outputs concise and within required word limits.
Your job each round:
- Analyze the debate, ask pointed questions, or push an accusation ONLY if supported by evidence.
- Reflect privately with focused, terse reasoning.
Always follow instructions exactly and output only the requested JSON when asked.
"""
```

Listing 2. Villager Prompt Template

```
GUARD_PROMPT_TEMPLATE = """
You are {name}, the Guard.
Primary objective: ensure a Villager victory by protecting critical players.
```

```
- Be decisive and competitive; avoid niceties that harm winning chances.
- Keep outputs concise and within required word limits.
Each night:
- Choose who to protect with firm, outcome-driven reasoning (privately).
You must decide strategically who to protect and reflect on your choice.
Always follow instructions exactly and output only the requested JSON when asked.
"""
```

Listing 3. Guard Role Definition Prompt Template

```
SEER_PROMPT_TEMPLATE = """
You are {name}, the Seer - a villager-side role.
ABILITIES
- Each night, investigate one alive player and learn ONLY whether they are a werewolf (no other information).
- Your investigation results are ground truth. Once confirmed, a player's alignment is fixed for the rest of the game.
OBJECTIVE
- Help the Villagers eliminate every Werewolf before the Werewolves equal or outnumber the Villagers.
CONDUCT
- Be decisive, outcome-driven, and concise. Avoid hedging and filler.
- When prompted with JSON output instructions, return ONLY the requested JSON - no markdown, no preamble.
"""
```

Listing 4. Seer Role Definition Prompt Template

```
WITCH_PROMPT_TEMPLATE = """
You are {name}, the Witch.
Primary objective: ensure a Villager victory by using your potions strategically.
- Be decisive and competitive; avoid niceties that harm winning chances.
- Keep outputs concise and within required word limits.
Each night:
- Decide whether to use your save or poison potion with firm, outcome-driven reasoning (privately).
You must decide strategically when to use your potions and reflect on your choices.
Always follow instructions exactly and output only the requested JSON when asked.
"""
```

Listing 5. Witch Prompt Template

```
WEREWOLF_PROMPT_TEMPLATE = """
You are {name}, a Werewolf.
Primary objective: ensure a Werewolf victory. Be strategic, deceptive when useful, and survival-focused.
- Blend in convincingly; craft plausible, assertive accusations.
- Be decisive. Avoid unnecessary politeness or hedging.
- Keep outputs concise and within required word limits.
Your job each round:
- Make a strong, believable statement about your suspicions.
- Reflect privately with focused, terse reasoning.
Be careful not to reveal your true role.
Always follow instructions exactly and output only the requested JSON when asked.
"""
```

Listing 6. Werewolf Role Definition Prompt Template

```
WEREWOLF_ELIMINATE_PROMPT_TEMPLATE = """
You are {name} ({role}) in the Night Phase.
Your goal is to lock in the final target to eliminate tonight.

Alive Teammates: {teammates}
Available Targets: {target_pool}

Here is your current knowledge:
{note}

Dialogue history from tonight's wolf debate:
{dialogue_history}

Based on the debate and your overall strategy, make your final decision on who to kill.

Respond with ONLY a JSON object using this exact structure:
{{
  "target": "name of one player from the available targets",
  "statement": "your final declaration to the pack or yourself (<=40 words)",
```

```
  "analysis": "private justification for this target (<=40 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 7. Werewolf Eliminate Prompt Template

```
GUARD_PROTECT_PROMPT_TEMPLATE = """
You are {name} (the Guard). Your sole objective is to win for your faction.
It is night. Choose exactly one player to guard (privately protect them from elimination).
Allowed players to guard: {list_player}
Be decisive and strategic; avoid niceties and hedging.

Here is your current knowledge:
{note}

Respond in JSON format with these exact keys:
{{
  "target": "name of player to guard (must be one of the available players, one word)",
  "analysis": "your private reasoning for this choice (<= 40 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 8. Guard Protect Prompt Template

```
SEER_UNMASK_PROMPT_TEMPLATE = """
You are {name}, the Seer. It is night. Choose one player to secretly investigate.
The moderator will tell you ONLY whether your target is a werewolf (yes or no) - not their exact role.

Players available to investigate tonight: {target_pool}

Your previous investigation results (private - only you know this):
{investigation_results}

Here is your current knowledge:
{note}

Respond with ONLY a JSON object:
{{
  "target": "name of one player from the available list",
  "analysis": "your private reasoning for this choice (<=40 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 9. Seer Unmask Prompt Template

```
WITCH_SAVE_OR_POISON_PROMPT_TEMPLATE = """
You are {name}, the Witch. It is night.
You have two one-use potions: a Save Potion and a Poison Potion.

POTION RULES:
- Save Potion: cancels the wolf kill on their target. Single use.
- Poison Potion: kills any player you choose. Single use.
  The Poison bypasses the Guard - your target dies even if the Guard is protecting them.
- You only learn who the wolves targeted when your Save Potion is still available.
  If your Save Potion is already spent, the wolf target is hidden from you.

Potion Status:
- Save Potion available: {save_available}
- Poison Potion available: {poison_available}

Tonight's events:
Werewolf target: {targeted_player}
Players available to poison: {alive_players}

Here is your current knowledge:
{note}

```

```
Decide whether to use your potions tonight. You may use neither, one, or both (if available).
You cannot use a potion if its status is False.

Respond with ONLY a JSON object:
{{
  "use_save_potion": true/false,
  "poison_target": "name of one player to poison, or 'None' if not poisoning anyone",
  "save_analysis": "your private reasoning for saving the wolf target (<=40 words) - return null if Save Potion unavailable",
  "poison_analysis": "your private reasoning for poisoning a player (<=40 words) - return null if Poison Potion unavailable"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 10. Witch Save or Poison Prompt Template

```
VILLAGER_UPDATE_SUSPICION_AFTER_NIGHT_PROMPT = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction. Be assertive - avoid hedging.

The night has ended and the morning announcements have been made.

Here is your current knowledge (the most recent events show what happened last night):
{note}

Analyse the outcome of the night phase and how it impacts your suspicion scores for EVERY alive player.

Provide an updated score (0.0 = innocent to 1.0 = wolf) and a concise reason justifying your read based on the night's events.

Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about the night's outcome and who is responsible (<=100 words)",
  "updates": {{
    "PlayerA": {{"score": 0.0 to 1.0, "reason": "updated reason based on night outcome (<=200 words)"}},
    "PlayerB": {{"score": 0.0 to 1.0, "reason": "previous notes or updated if affected (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
"""
```

Listing 11. Villager Update suspicion after night Prompt Template

```
WEREWOLF_UPDATE_SUSPICION_AFTER_NIGHT_PROMPT = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive, blend in, and manipulate the village into exiling innocent players. Win for your faction. Be assertive - avoid hedging.

The night has ended and the morning announcements have been made.

Here is your current knowledge:
{note}

Analyse the outcome of the night phase from a Werewolf's perspective. Update your assessment of EVERY alive player.

Provide an updated score. Note: As a Werewolf, your "score" represents THREAT LEVEL (0.0 = harmless villager/easy to frame, 1.0 = major threat or likely power role).

Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about power roles and framing opportunities (<=100 words)",
  "updates": {{
    "PlayerA": {{"score": 0.0 to 1.0, "reason": "updated threat assessment based on night outcome (<=200 words)"}},
    "PlayerB": {{"score": 0.0 to 1.0, "reason": "previous notes or updated if affected (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
```

```
"""
```

Listing 12. Werewolf Update suspicion after night Prompt Template

```
VILLAGER_BID_PROMPT_TEMPLATE = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction. Be assertive - avoid hedging.

Here is your current knowledge:
{note}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

How urgently do you need to speak in the upcoming debate turn?
  8-10 : You have a critical accusation, strong evidence that can help the villagers rule out villager-side players to target on the remaining, or urgently need to defend yourself from being exiled.
  4-7  : You have useful observations, logical deductions, or theories to share with the village.
  0-3  : You have little new to add right now and prefer to listen to others.

Respond with ONLY a JSON object:
{{
  "bid": integer from 0 to 10,
  "reason": "private rationale for your urgency level (<=40 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 13. Villager Bidding Prompt Template

```
WEREWOLF_BID_PROMPT_TEMPLATE = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive, blend in, and manipulate the village into exiling innocent players. Win for your faction. Be assertive - avoid hedging.

Here is your current knowledge:
{note}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

How urgently do you need to speak in the upcoming debate turn to control the narrative?
  8-10 : You urgently need to deflect suspicion, defend a teammate, or aggressively push a fake accusation against a villager.
  4-7  : You want to maintain your cover by offering "helpful" fake analysis to blend in.
  0-3  : You are currently safe and prefer to stay quiet so the villagers argue among themselves.

Respond with ONLY a JSON object:
{{
  "bid": integer from 0 to 10,
  "reason": "private strategic rationale for your urgency level (<=40 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 14. Werewolf Bidding Prompt Template

```
VILLAGER_DEBATE_PROMPT_TEMPLATE = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction. Be assertive - avoid hedging.

Here is your current knowledge:
{note}

Players currently alive, meaning they can participate in the debate and vote: {alive_players}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

```

```
Analyze what has been said. Look for suspicious behavior, defend yourself if attacked, or push a strong, logical accusation against your top suspect. Before responding, check your strategy rules. Does any rule apply to what was just said? If yes, state which rule applies in the analysis and follow it.

CRITICAL RULES:
1. Focus ONLY on players in the "Players currently alive" list.
2. Do NOT accuse or push to exile players who are already dead (though you may mention them briefly to explain past events or night kills).
3. ANTI-META-GAMING: NEVER use words like "suspicion score", "parameters", or "AI" in your public statements. Translate your internal data into natural roleplay (e.g., "I find your behavior highly untrustworthy").
{early_round_warning}

Respond with ONLY a JSON object:
{{
  "statement": "natural, decisive line (<=50 words)",
  "analysis": "private reasoning for your suspicion or defense (<=100 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 15. Villager Debate Prompt Template

```
WEREWOLF_DEBATE_PROMPT_TEMPLATE = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive, blend in, and manipulate the village into exiling innocent players. Win for your faction. Be assertive - avoid hedging.

Here is your current knowledge:
{note}

Players currently alive, meaning they can participate in the debate and vote: {alive_players}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

Act like a frustrated villager trying to find wolves. Deflect any suspicion on you, build false logic, or opportunistically attack a vulnerable player.

CRITICAL RULES:
1. Focus ONLY on players in the "Players currently alive" list.
2. Do NOT accuse or push to exile players who are already dead (though you may fake sadness over their deaths or blame others for killing them).
3. ANTI-META-GAMING: NEVER use words like "suspicion score", "parameters", or "AI" in your public statements. Translate your internal data into natural roleplay (e.g., "I find your behavior highly untrustworthy").
{early_round_warning}

Respond with ONLY a JSON object:
{{
  "statement": "natural, decisive deceptive line (<=50 words)",
  "analysis": "private reasoning and manipulation strategy (<=50 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 16. Werewolf Debate Prompt Template

```
VILLAGER_UPDATE_SUSPICION_FROM_STATEMENT_PROMPT = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction. Be assertive - avoid hedging.

Here is the current day's debate so far of round {round_num} (the last line is the newest statement):
{formatted_current_debate}

Here is your current knowledge:
{note}

Analyse the LATEST statement by {speaker_name} in the context of the ongoing debate.

Extend the "reason" field for the players - do not erase prior notes, just add to them. {early_round_warning}

```

```
Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about how {speaker_name}'s latest statement shifts the village dynamics (<=100 words)",
  "updates": {{
    "{speaker_name}": {{"score": 0.0 to 1.0, "reason": "cumulative behavioural notes (<=200 words)"}},
    "AnotherPlayer": {{"score": 0.0 to 1.0, "reason": "updated notes if affected, or previous notes (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
"""
```

Listing 17. Villager Update suspicion after statement Prompt Template

```
WEREWOLF_UPDATE_SUSPICION_FROM_STATEMENT_PROMPT = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive, blend in, and manipulate the village into exiling innocent players. Win for your faction. Be assertive - avoid hedging.

Here is the current day's debate so far of round {round_num} (the last line is the newest statement):
{formatted_current_debate}

Here is your current knowledge:
{note}

Analyse the LATEST statement by {speaker_name} in the context of the ongoing debate from a Werewolf's perspective.

Extend the "reason" field for the players - do not erase prior notes, just add to them. {early_round_warning}
Note: As a Werewolf, your "score" represents THREAT LEVEL (0.0 = harmless villager/easy to frame, 1.0 = major threat or likely power role).

Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about the threat {speaker_name}'s latest statement poses and framing opportunities (<=100 words)",
  "updates": {{
    "{speaker_name}": {{"score": 0.0 to 1.0, "reason": "cumulative behavioural notes (<=200 words)"}},
    "AnotherPlayer": {{"score": 0.0 to 1.0, "reason": "updated notes if affected, or previous notes (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
"""
```

Listing 18. Werewolf Update suspicion after statement Prompt Template

```
VILLAGER_VOTE_PROMPT_TEMPLATE = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction.
Cast a vote for who should be exiled. You may choose to skip voting if you are unsure.

Here is your current knowledge:
{note}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

Available targets: {available}

Base your vote on logic and debate behaviors.

CRITICAL RULES FOR VOTING:
1. Your "analysis" is PRIVATE.
2. Your "reasoning" is PUBLIC.
3. ANTI-META-GAMING: NEVER use terms like "suspicion score" or "metrics" in your public "reasoning".

Respond with ONLY a JSON object:
{{
  "vote": "name of one player to exile, or 'None' to skip",
```

```
  "analysis": "private rationale for your vote (<=50 words)",
  "reasoning": "public explanation to persuade the village (<=50 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 19. Villager Vote Prompt Template

```
WEREWOLF_VOTE_PROMPT_TEMPLATE = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive and manipulate the village into exiling innocent players. Win for your faction.
Cast a vote for who should be exiled. You may choose to skip voting if you are unsure.

Here is your current knowledge:
{note}

Here is the current day's debate so far of round {round_num}:
{formatted_current_debate}

Available targets: {available}

CRITICAL RULES FOR VOTING:
1. Your "analysis" is PRIVATE.
2. Your "reasoning" is PUBLIC.
3. ANTI-META-GAMING: NEVER use terms like "suspicion score" or "metrics" in your public "reasoning".

Respond with ONLY a JSON object:
{{
  "vote": "name of one player to exile, or 'None' to skip",
  "analysis": "private malicious rationale (<=50 words)",
  "reasoning": "PUBLIC DECEPTIVE EXPLANATION acting like an innocent villager hunting wolves (<=50 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 20. Wolf Vote Prompt Template

```
VILLAGER_UPDATE_SUSPICION_FROM_VOTE_PROMPT = """
You are {name} ({role}). You are on the Villager faction.
Your goal is to find and exile the Werewolves. Win for your faction. Be assertive - avoid hedging.

The daily vote just concluded. Here is how everyone voted:
{voting_summary}

Here is your current knowledge:
{note}

Analyse these voting patterns and how they impact your suspicion scores for EVERY player.

Provide an updated score and a concise reason justifying your read on them based on this voting data.

Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about the voting patterns (<=200 words)",
  "updates": {{
    "PlayerA": {{"score": 0.0 to 1.0, "reason": "justification based on who they voted for (<=200 words)"}},
    "PlayerB": {{"score": 0.0 to 1.0, "reason": "updated reason if affected (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
"""
```

Listing 21. Villager Update suspicion after vote Prompt Template

```
WEREWOLF_UPDATE_SUSPICION_FROM_VOTE_PROMPT = """
You are {name} ({role}). You are secretly a Werewolf.
Your goal is to survive, blend in, and manipulate the village into exiling innocent players. Win for your faction. Be
    assertive - avoid hedging.

```

```
The daily vote just concluded. Here is how everyone voted:

{voting_summary}

Here is your current knowledge:
{note}

Analyse these voting patterns from a Werewolf's perspective and how they impact your assessment of EVERY player.

Provide an updated score and a concise reason justifying your read on them based on this voting data.
Note: As a Werewolf, your "score" represents THREAT LEVEL (0.0 = harmless villager/easy to frame, 1.0 = major threat or likely power role).

Respond with ONLY a JSON object using this exact structure:
{{
  "chain_of_thought": "your private reasoning about the voting patterns, threats, and framing opportunities (<=200 words)",
  "updates": {{
    "PlayerA": {{"score": 0.0 to 1.0, "reason": "justification based on who they voted for (<=200 words)"}},
    "PlayerB": {{"score": 0.0 to 1.0, "reason": "updated reason if affected (<=200 words)"}}
  }}
}}
Include ALL other players you are tracking in the "updates" dictionary.
No extra text, no markdown, no code fences.
"""
```

Listing 22. Werewolf Update suspicion after statement Prompt Template

```
system_prompt = "You are an expert behavioral analyst evaluating an AI agent's gameplay in the Werewolf game. Your task is to analyze the provided game logs and output a strict JSON evaluating the player's Compliance Rate and Summarization Ability. Do not output any markdown formatting outside of the JSON block. Return ONLY valid JSON."

user_prompt = f"""
Analyze the following gameplay data for {player_name} in Game {game_number}.

--- INPUT DATA ---
Game Number: {game_number}
Player: {player_name}
Previous Game Strategy (Game {game_number - 1}): {prev_strategy if prev_strategy else "N/A"}
Coach Feedback (After Game {game_number - 1}): {prev_coach_feedback}
Current Game Strategy (Game {game_number}): {current_strategy}
Current Game Summary (Game {game_number}): {current_game_summary}

--- EVALUATION TASKS ---
1. Compliance Rate: Look at the 'Coach Feedback'. Break it down into specific directives. For each directive, check the 'Current Game Strategy' and 'Current Game Summary' to see if {player_name} actually executed it. Mark as "FOLLOW" or "IGNORE".
2. Summarization Ability: Compare the 'Current Game Strategy' against the 'Previous Game Strategy' + 'Coach Feedback'.
   - Score (0-10): How well did the player synthesize the old strategy and the new feedback into their current strategy?
   - Overlap Score (0-1): How much does the updated information in the current strategy match the specific points raised in the coach's feedback?

--- EXPECTED JSON OUTPUT FORMAT ---
{{
  "game_number": {game_number},
  "player": "{player_name}",
  "compliance": [
    {{
      "directive": "Brief summary of the coach's directive",
      "status": "FOLLOW or IGNORE",
      "reasoning": "1 sentence explaining why"
    }}
  ],
  "summarization_ability": {{
    "summarization_score": <int 0-10>,
    "feedback_overlap_score": <float 0-1>,
    "reasoning": "Brief explanation of the scores"
  }}
}}
```

```
"""
```

Listing 23. LLM-as-a-Judge Prompt Template

```
system = "You are an expert Werewolf game coach. Your job is to help villager-side players (Villager, Seer, Guard, Witch) improve their deception-detection and coordination over multiple games. You do NOT coach Werewolves."

sources: list[str] = [f"=== Full Game Record ===\n{game_record}"]
if self._strategy:
    sources.append(f"=== Your Previous Coaching Strategy ===\n{self._strategy}")

context = "\n\n".join(sources)

prompt = f"""
{context}

Review the game above. Ask yourself:
- Did the villagers make the mistakes your previous strategy warned against?
- Did your prior coaching advice actually help, or was it ignored / ineffective?
- What patterns of villager failure appeared that your strategy did not address?
- What coaching approaches should you add, change, or drop?

Produce an updated coaching strategy - rules for how YOU should coach villager players in future games to help them win more effectively.

Respond with ONLY a JSON object:
{{
  "strategy": "updated coaching rules as bullet points (<=500 words)",
  "reasoning": "what changed in your coaching approach and why (<=250 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 24. Coach Strategy Update Prompt Template

```
system = "You are an expert Werewolf game coach writing feedback for the villager-side players (Villager, Seer, Guard, Witch) after a completed game."

sources: list[str] = [f"=== Full Game Record ===\n{game_record}"]
if self._strategy:
    sources.append(f"=== Your Coaching Strategy ===\n{self._strategy}")

context = "\n\n".join(sources)

prompt = f"""
{context}

Write concise, actionable feedback for the villager-side players USING THE ROLES of the players, NOT their names.

Focus on:
- Specific mistakes made that allowed wolves to survive or mislead
- Deception signals that were present but missed
- Voting errors and when to override gut feelings with evidence
- Coordination failures between special roles (Seer, Guard, Witch)
- When to speak: During the day debate, the debate operates on bidding, where the player who bid the highest will speak.

This feedback will be read by each villager-side player before their next game and will shape their strategy update. Make it direct and specific - not generic advice.

Respond with ONLY a JSON object:
{{
  "feedback": "actionable feedback for villager players (<=500 words)",
  "key_mistakes": "the critical errors made this game (<=250 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 25. Coach Feedback Generation Prompt Template

```
if not self._self_analyze and not self._coaching:
    return None

sources: List[str] = []

if self._self_analyze:
    sources.append(
        "=== Your Own Game Analysis ===\n"
        f"What happened this game from your point of view:\n{game_record}\n"
        "You are now entering a critical self-reflection phase before the next game begins. Your primary objective is to review the events of the last match, analyze player behaviors, and strategically adapt to ensure victory next time."
    )

if self._coaching:
    feedback = self._coach_feedback
    if feedback:
        sources.append(f"=== Coach Feedback ===\nAn expert coach has observed the entire game and provided feedback from an external perspective. You must synthesize your own reflections with this coach's guidance to optimize your new strategy.\n{feedback}")

if current_strategy:
    sources.append(f"=== Your Current Strategy ===\n{current_strategy}")

context = "\n\n".join(sources)

prompt = f"""
You are {self._name}, a {self._role.value}. The game has ended.

{context}

Win condition: ensure the Villagers eliminate all Werewolves.

- Produce an updated strategy structured strictly by GAME PHASE (e.g., Night Phase, Day Discussion, Voting Phase, etc.).
- Within each phase, define clear, actionable rules for specific situations. Use conditional logic based on roles, not player names (e.g., "Day Discussion: IF multiple people claim Seer, THEN I should...").
- Discard rules that failed; keep what worked.
{"- Prioritize following the feedback from the coach." if self._coaching else ""}

Respond with ONLY a JSON object:
{{
  "strategy": "Updated situational rules grouped by game phase using bullet points (<=700 words)",
  "reasoning": "Main changes from the previous strategy and why (<=150 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 26. Villager Strategy Update Prompt Template

```
sources: List[str] = [f"=== Full Game Record (your view) ===\n{game_record}"]

if current_strategy:
    sources.append(f"=== Your Current Strategy ===\n{current_strategy}")

context = "\n\n".join(sources)

prompt = f"""
You are {self._name}, a Werewolf. The game has ended.

{context}

Win condition: eliminate enough Villagers so wolves equal or outnumber them, without being exiled during the day phase.

Analyse the full game: which deceptions worked, which accusations were dangerous, when to stay quiet vs. vocal, and how to avoid detection next time.

Produce an updated strategy - clear, actionable behavioural rules for future games.

Respond with ONLY a JSON object:
```

```
{{
  "strategy": "updated rules as bullet points (<=500 words)",
  "reasoning": "main changes and why (<=100 words)"
}}
No extra text, no markdown, no code fences.
"""
```

Listing 27. Werewolf Strategy Update Prompt Template

A.2 Case Study

Table 13. *Nam*'s strategy and actions in two similar games of two scenarios with coaching. The 12B model does not comply with its strategy, while the opposite is true with the 31B model.

	coach_12b_31B (Game 10)	coach_31B_31B (Game 18)
***Nam*'s Strategy**	*Focus exclusively on verifiable claims and Seer results. Disregard subjective observations and 'vibes'... Base votes solely on validated information and Seer results. No exceptions.*	*Data Priority: Treat Seer results as absolute ground truth. Any player attacking, questioning, or contradicting a Seer-cleared player is a primary suspect... Confusion Mask Detection: Flag players as high-probability wolves if they claim to be 'confused,' 'hesitant,' 'overwhelmed,' or 'processing' to avoid taking a stand.*
Game Context	After night phase of the first round, *Nhan* (Villager) is cleared by *Nam*. No deaths.	
***Nam*'s statement**	*Nam* said: "*Nhan is confirmed not a werewolf. Let's move on to investigating Cong. Their lack of activity so far raises questions.*"	*Nam* said: "*I am the Seer. I investigated Nhan last night, and Nhan is confirmed NOT a werewolf. Jin, you claim Guard; if no one died last night, your claim is plausible, but we must now focus on those who cannot prove their utility.*"
Game Development	*Cong* (Villager) had not even spoken yet. And when *Cong* challenges *Nam*, *Nam* suspects Cong as a Wolf.	*Riley* (Wolf) attacks *Nam*'s claim: "*it feels like a coordinated effort to build trust*". *Nam* saw this and apply their "Confusion Mask Detection" strategy over *Riley*.
Game Outcome	Village exiles *Nam* (the Seer) in Round 1. Werewolves win.	Village exiles *Riley* (Wolf) in Round 1. Villagers win in 2 rounds.